Course: UCCS CS 3300 Into to Software Engineering

Date: 4/26/2026

Project Ttitle: Github.io

Developer's (Future Programmer's) Document

Description:

GameHub.io is a centralized adless gaming platform, which allows users to play their favorite games, track their scores, and be ranked against their friends and peers. Many current websites don't include nostalgic games from our childhood like Hangman and Snake or have features for users to easily track their progress and compare their performance against their friends. GameHub.io solves all of that by creating a centralized website with everyone's favorite nostalgic games, and friendly competition amongst friends, with an ad-free experience.

Configuration Management:

How did we manage our software configuration –

As a team, our primary tactic for managing our software configuration was to limit the number of dependencies, moving parts, and external tools as much as possible. Throughout the production of this software, we stuck to one version of all our languages, tools, frameworks, and dependencies. We also utilized importing dependencies through API calls live as opposed to trying to store, manage, and package dependencies for the program. To ensure the flow of production was smooth and consistent for all developers, we set up identical environments in VS Code. Each environment was setup with the same server extension, language versions, and code formatting tools to create a consistent developer environment.

License and open-source details –

Our product is licensed under an MIT License. This is possible because all the code, art, and functionality of our website and games is originally created by our team of developers. We have games on our website that have been heavily inspired by games that exist in public domain such as –

Original Game Name	Original Author / Publisher	Why it's usable?
Snake	Nokia	Nokia only holds copyright / patents on specifically their code, functionality, and art used for their game not the

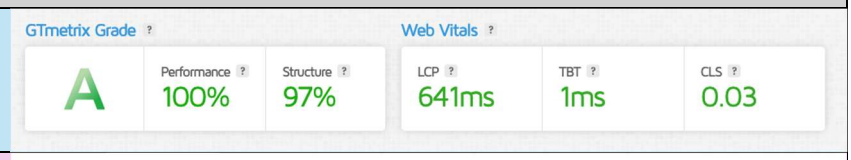
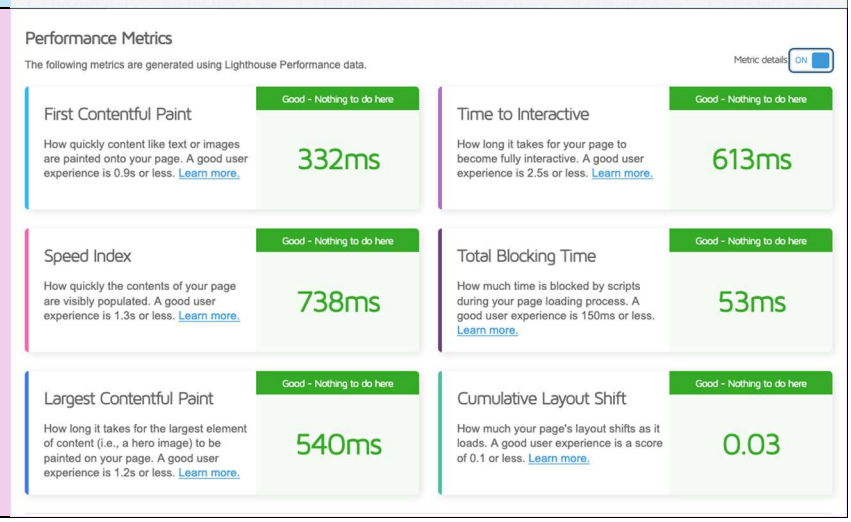
		idea or general mechanics of snake.
Hangman	Undefined	There is no one owner of hangman, therefore if the code is originally produced there are no copyright concerns.

Our 3rd party dependencies are listed below –

Dependency	Version	License
Firebase (JavaScript SDK)	v12.12.1	Apache License 2.0
SweetAlert2	v11.26.24	MIT License

Metrics –

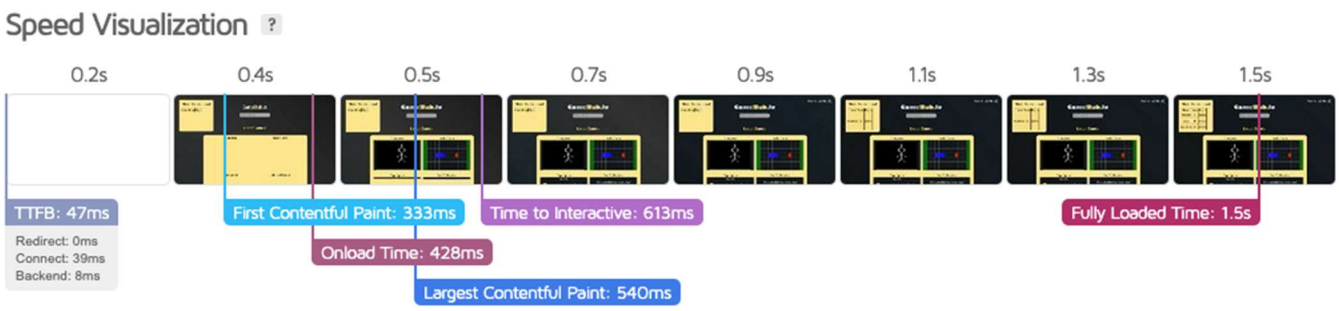
Using an industry recognized tool *GTMetrix* we were able to collect standardized performance metrics for our website.

Test	Output	Test Screenshot
GTMetrix Overall	Grade: A	
Performance Metrics	Average of **45% faster** than the minimum acceptable speeds	

Structure Metrics	Grade: 97%	

Show Audits Relevant to: All | FCP | LCP | TBT | CLS

IMPACT	AUDIT
Med-Low	Serve static assets with an efficient cache policy
Med-Low	Use a Content Delivery Network (CDN)
Low	Avoid enormous network payloads LCP
Low	Reduce JavaScript execution time TBT
Low	Avoid chaining critical requests FCP LCP
Low	Remove duplicate modules in JavaScript bundles TBT
Low	Avoid long main-thread tasks TBT
Low	Serve images in next-gen formats
Low	Avoid non-composited animations CLS
Low	Minify JavaScript FCP LCP
Low	Reduce unused JavaScript LCP
Low	Properly size images



Through our team's planning of our website structure, we have managed to ensure the loading order of assets is optimized to prevent any waiting times for previous assets. While our frame data is great there are some things to note as far as improvement. The styles and assets for our website could use some small restructuring to make sure that we are only loading necessary data for each webpage. This could be done by both refactoring our CSS file to separate files for each section of the website as well as redesigning the "Search UI" functionality to predict the static assets that will be needed for display before they are needed.

Code Summary –

Date: 2026-04-27 18:51:55

Directory: GameHub.io/*

Total: 29 files, 2465 codes, 407 comments, 541 blanks, all 3413 lines

Languages –

language	files	code	comment	blank	total
JavaScript	12	1,349	202	291	1,842
HTML	12	566	175	156	897
PostCSS	1	475	30	77	582
JSON	3	73	0	16	89
Markdown	1	2	0	1	3

Directories –

path	files	code	comment	blank	total
.	29	2,465	407	541	3,413
. (Files)	3	29	0	3	32
web_page	26	2,436	407	538	3,381
web_page (Files)	3	832	95	140	1,067
web_page/games	13	1,118	147	220	1,485
web_page/games/hangman	3	307	34	44	385
web_page/games/path_to_paradise	3	159	20	39	218
web_page/games/snake	4	369	59	107	535
web_page/games/test_game	1	21	6	2	29
web_page/games/type_sprint	2	262	28	28	318
web_page/htmls	6	309	119	115	543
web_page/scripts	4	177	46	63	286

System Requirements –

The following minimum system specifications are required to access and operate the GameHub.io application. These requirements apply to end users accessing the system via a web browser.

Operating System: Windows 10 or later, macOS 11 (Big Sur) or later, or a modern Linux distribution.

Web Browser: Google Chrome (v100+), Mozilla Firefox (v100+), Microsoft Edge (v100+), or Safari (v15+). Internet Explorer is not supported.

Internet Connection: A stable broadband connection with a minimum speed of 5 Mbps is recommended for optimal performance.

Display: A resolution of 1920×1080 or greater is recommended.

Browser Settings: JavaScript and cookies must be enabled. Pop-up blockers should be configured to allow pop-ups from “cs3300-groupproject.web.app”.

Mobile Devices: The application is compatible with iOS 15 or later and Android 10 or later, accessed via the device's default or supported browser.

Reviewing Requirements –

As a team we managed to build, test, and implement **22/22** of our *minimum viable product* requirements. These requirements include but are not limited to, outlining, planning, and programming the primary website pages and navigation throughout, designing, planning and programming both visual and logical functionality for 4 games (Hangman, Snake, TypeSprint, and Path to Paradise), designing and programming logical ties between score tracking for each applicable game and the live leaderboards visible to all users and creating a dynamic, searchable, ad-free space for users to play and compete in these games while saving their data by signing in to our platform. While our team was successful in delivering our *MVP,* we only managed to deliver **28/33** total requirements we outlined for this project.

2.1.2	Implement light/dark themes for games	whole team	Brandon Sith	Ver 2.1	18-Apr	N/A
2.2.1	Design/Build games 4-6	whole team	Brandon/David	Ver 2.2	22-Apr	N/A
2.2.2	Implement canvas display for games 4-6	whole team	Brandon/Trey	Ver 2.2	22-Apr	N/A
2.2.4	Implement password reset	whole team	Trey/Azam	Ver 2.2	22-Apr	N/A
3.0.1	Implement server run multiplayer game	whole team	Whole Team	Ver 3.0	22-Apr	N/A

Diving into the requirements that we set for ourselves we can see clearly that there were no requirements that fell under *minimum viable product* that were not completed. We have 5 requirements that did not get implemented including designing/programming/implementing games 5 & 6 and their canvas’, password resetting, light/dark theme toggles for website & games, and programming/implementing a server run multiplayer game. As these requirements were not necessary for our *MVP* it is acceptable for these to be unimplemented at this time. We can consider all necessary requirements to be implemented and fulfilled.

Software Verification -

Our team planned for 18 unit test, 12 integration tests and 3 system tests, totaling 33 tests for 33 requirements. Out of these tests we completed 16/18 unit tests, 10/12 integration tests, and 2/3 system tests because of uncompleted requirements.

Test Type	Number Tests Planned	Number of Tests Executed
Unit Test	18	16
Integration Test	12	10
System Test	3	2

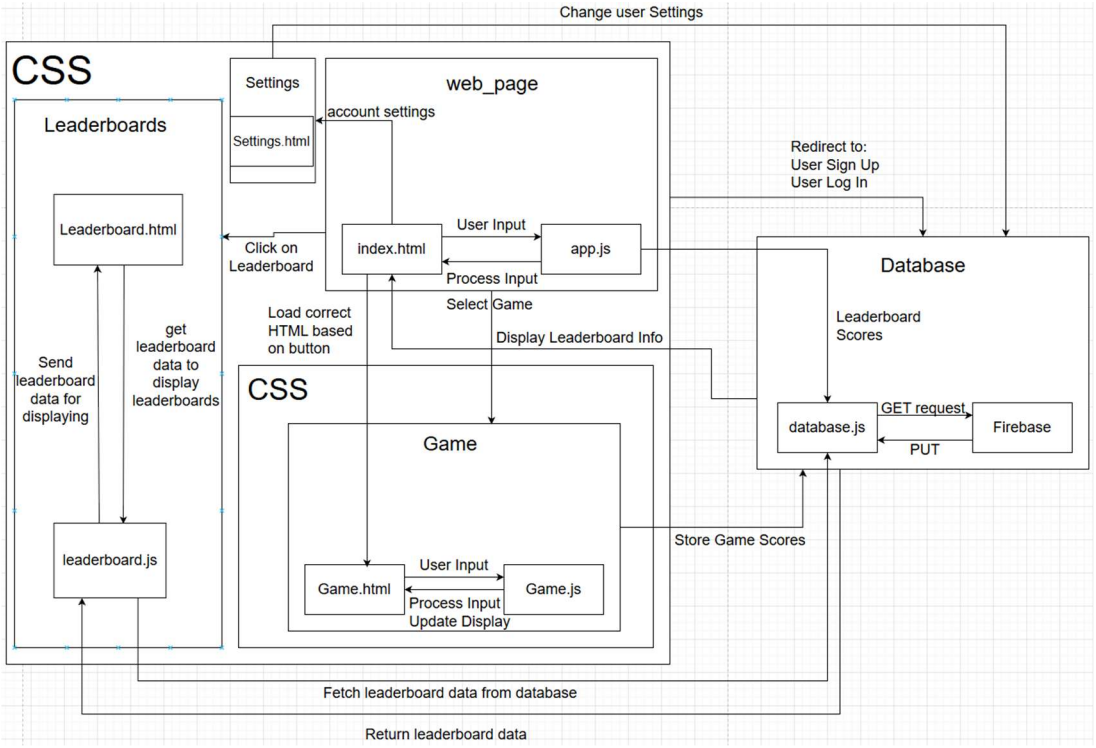
This lets us deduce that our team had a requirement implementation and testing ratio of 28/32 or 84% of total requirements.

Current Code Condition -

All the things we flagged during development have been fixed. That being said, there are a few things we would have liked to implement/polish that we were unable to do. For our requirements, our goal was to have 4-6 games on the website. We completed this to the bare minimum with 4 games. We would have liked to have more, but time was a limitation for us. Similar to this last requirement, we wanted to have canvas displays for 4-6 games. Since we only have 4 games, this requirement is also incomplete. Another requirement we were unable to fulfill was the password reset feature. This feature ended up getting expanded a little bit. Our hopes were to be able to edit account info such as username, password, and display name. Unfortunately, we didn't have time to get this one done either. Some other goals we fell short of were having a like/dark theme for the games and website. Currently we only have a dark theme because the natural color scheme of the website is darker colors. The last requirement we didn't complete was a server run multiplayer game. This game was kind of a stretch for us in the first place, so there's no surprise that we didn't get to it. We originally had this as an optional requirement to complete only if we had time.

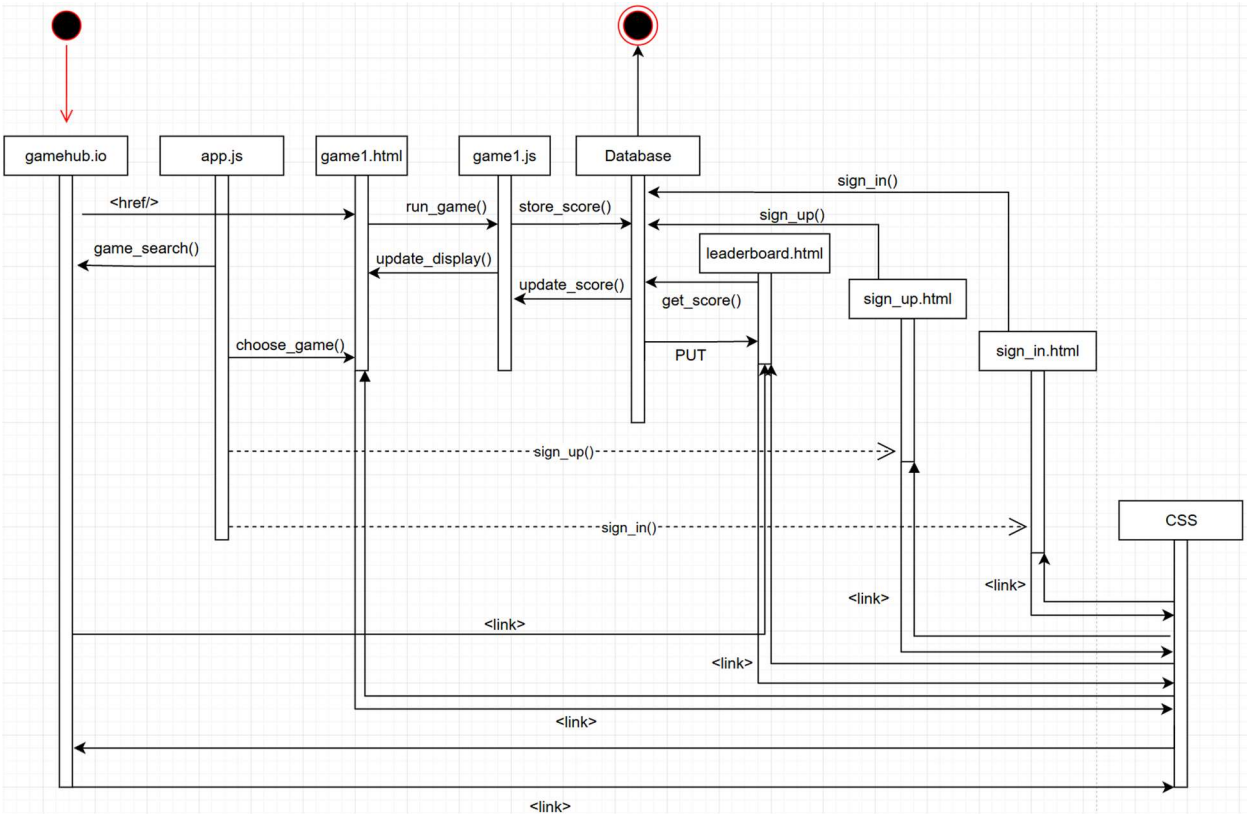
All of our games and website elements are pretty polished. We don't have any known bugs at the moment, and all the issues with the website stem from the incomplete requirements stated above. Even though we didn't complete 100% of our requirements, the website is still operational. These missing requirements do not hinder the base functionality of the website.

Architectural Diagram: Logical



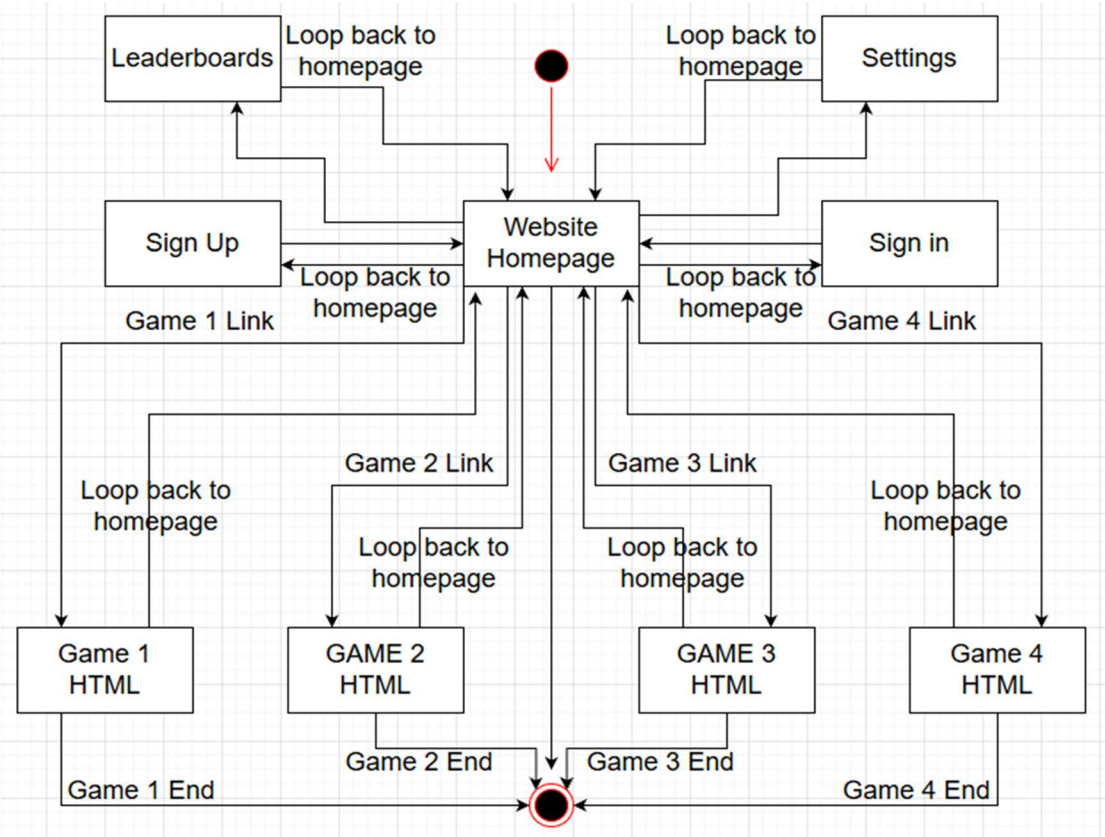
The Webpage is wrapped in a large CSS file. The homepage of the website is run by app.js. App.js does everything from processing link clicks, displaying the leaderboards, and dynamically changing based off user input (hovering over games or searching in the search bar). When you click on a game link, you will be brought to another html page. This html page is run by the game's JavaScript files. When the game ends, the game file will submit the score to the database. Each game is wrapped in its own CSS file. This CSS file is much smaller than the main one and is only used for game specific elements. From the homepage, you can visit the leaderboards page by clicking on the leaderboard preview. This brings you to the leaderboards html which is run by the leaderboards JavaScript file. This JS file requests data from the database to be displayed on the html. From the main page, if you are signed in, you can go to a settings page, and if you are not signed in, you can sign in/sign up through a sign in/sign up page. All user data including username, passwords, and game specific scores are stored in the database.

Software Design Diagram: Sequence



The Diagram shows the sequence in which our code is organized. All the HTML files are formatted by our CSS file. We start on the homepage (gamehub.io) which the user can interact with through app.js. App.js processes user input and displays all the elements of the page. From the homepage, you can select a game. Clicking the game redirects the user to the game HTML which is run by the game's JS files. When the game ends, the scores are submitted to the database. The user can then return to the homepage. From the homepage, the user can visit the leaderboard page by clicking on the leaderboard. The leaderboards HTML is run by the leaderboards JS which requests data to display from the database. If the user is not signed in, they can create an account or sign in using the sign in/sign up pages. These pages operate by requesting and putting data such as usernames and passwords into the database.

Perspective Model Diagram: Activity



This activity diagram gives an idea of how the user might navigate the webpage. The user starts at the homepage. From there, the user can sign up or sign in. Once logged in, the user can click on the leaderboards from the homepage. If they would like to edit their account, they can also visit the settings page. All these pages are looped back to the homepage. When the user is ready to play a game, they can select 1 of 4 games. Clicking on the game picture will redirect the user to different game pages. After playing the game, the user can decide to exit the website or go back to the home page. There, they can select a new game to play, view the updated leaderboards, or exit the website.

Software Tools Used:

Comprehensive Table:

Language / Tool	Version
JavaScript	ECMAScript 2025 (16th Edition)
CSS (Cascading Style Sheets)	Living Standard (No Version Iterations)
HTML5	Living Standard (No Version Iterations)
Firebase SDK	v12.12.1
SweetAlert2	v11.26.24